

COMPUTEMESH

Vollständiger technischer, wirtschaftlicher und Launch-Blueprint

THE INTERNET IS YOUR GPU

Version 1.0 | August 2026

Ziel: Viele kleine und große GPUs zu einer virtuellen KI-Infrastruktur verbinden, damit Nutzer Modelle ausführen können, die auf ihrer eigenen Hardware niemals Platz hätten.

Produktvision • Architektur • Sharding • MoE • Scheduler • Sicherheit • Verifikation • Payment • Roadmap • Launch

0. Executive Summary

ComputeMesh ist als offene Distributed-AI-Execution-Layer gedacht. Das Kernversprechen lautet: Ein Nutzer mit nur 8 GB VRAM soll über ein weltweites Netz aus Consumer-GPUs, Workstations und professionellen Beschleunigern ein Modell starten können, dessen Speicher- und Rechenbedarf ein Vielfaches der eigenen Hardware beträgt.

Nicht die GPU ist das Produkt. Das Produkt ist der Zugriff auf ein virtuelles, verteiltes KI-System.

Das Projekt wird bewusst in zwei Ebenen getrennt: Erstens muss die technische Kernfrage gelöst werden - performante, fehlertolerante, heterogene LLM-/MoE-Inferenz über reale Netzwerke. Zweitens wird darum ein Marktplatz mit Reputation, Verifikation, Abrechnung und später dezentraler Governance gebaut.

Die drei Kernversprechen

Zielgruppe	Versprechen
Nutzer	Run models bigger than your computer.
Provider	Contribute your GPU. Earn compute.
Unternehmen	One API. A global pool of heterogeneous AI compute.

Die drei Go/No-Go-Tore

Gate	Frage	Konsequenz
1 - Technische Machbarkeit	Können heterogene GPUs ein Modell automatisch gemeinsam ausführen?	Wenn nein: Architektur ändern.
2 - Internet-Performance	Ist die Inferenz über echte Internetverbindungen interaktiv brauchbar?	Wenn nein: Regionalcluster, Batch oder MoE priorisieren.
3 - Wirtschaftlichkeit	Ist Cost/Token konkurrenzfähig?	Wenn nein: Marketplace nicht als Hauptprodukt positionieren.

A. Inhaltsübersicht

Teil	Thema	Abschnitte
Teil I	Vision & Produkt	1-5
Teil II	Node, Benchmark & Compute Units	6-9
Teil III	Model Registry, Sharding & Topologie	10-15
Teil IV	Internet-Native MoE & Neural CDN	16-20
Teil V	Fehlertoleranz, Datenschutz & Sicherheit	21-25
Teil VI	Verifikation, Reputation & Payment	26-31
Teil VII	UX, API & Softwarearchitektur	32-39
Teil VIII	Tests & Entwicklungsmeilensteine	40-53
Teil IX	Launch & Wachstum	54-57
Teil X	IP, Team, KPIs & Execution	58-65

Die Nummerierung entspricht dem Entwicklungs-Blueprint und ist bewusst fein granular gehalten, damit daraus direkt Tickets, Epics, ADRs und technische Arbeitspakete abgeleitet werden können.

1. Produktdefinition

ComputeMesh soll aus kleinen und großen GPUs eine gemeinsame, virtuelle KI-Infrastruktur bilden. Ein Teilnehmer kann gleichzeitig Nutzer und Provider sein. Lokale Hardware wird dabei nicht ersetzt, sondern mit Netzwerkressourcen ergänzt.

LOCAL HARDWARE + NETWORK HARDWARE + DATACENTER HARDWARE
↓
VIRTUAL AI MACHINE

Das strategische Ziel ist nicht, einen weiteren GPU-Cloudanbieter zu bauen, sondern eine Ausführungsschicht, die die physische Fragmentierung der Hardware für den Nutzer abstrahiert.

2. Die zwei Benutzergruppen

Consumer

Beispiel: RTX 3060 mit 8 GB VRAM. Der Nutzer möchte ein 70B-, 400B- oder später noch größeres Modell verwenden. ComputeMesh beschafft automatisch fehlende Modellsegmente und Rechenkapazität.

Provider

Provider können eine einzelne Consumer-GPU, eine Workstation, ein ehemaliges Mining-Rig, einen A100/H100-Server oder professionelle Cluster bereitstellen. Sie legen Verfügbarkeit, Power-Limit, Mindestpreis und Datenschutzklasse fest.

3. Das zentrale Produktversprechen

YOU HAVE 8 GB. THE MODEL NEEDS 300 GB. RUN IT ANYWAY.

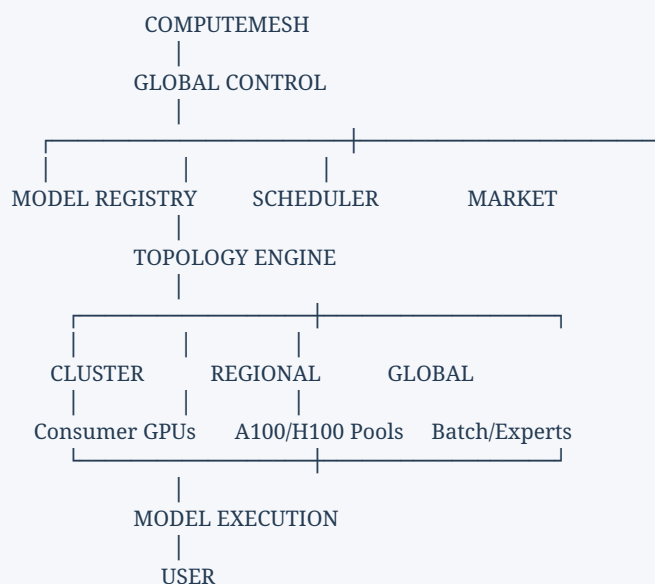
Die Nutzererfahrung muss so weit abstrahiert werden, dass VRAM-Planung, Sharding, Node-Auswahl, Parallelisierungsstrategie und Failover im Hintergrund stattfinden. Der Nutzer wählt ein Modell und sieht nur, ob ausreichend Netzwerkressourcen bereitstehen.

4. Was ComputeMesh ausdrücklich nicht ist

- kein Bitcoin-Miner
- kein Token-/ICO-Projekt
- kein Kubernetes-Frontend
- kein weiteres Ollama
- kein reiner GPU-Stunden-Marktplatz
- kein einzelnes Foundation Model
- kein System, das beliebigen Kundencode unisoliert auf Heim-PCs ausführt

ComputeMesh = Distributed AI Execution Layer

5. Gesamtarchitektur



Die Architektur trennt Control Plane und Data Plane. Das Control Plane verwaltet Identitäten, Modelle, Hardwareprofile, Topologie, Preise und Reputation. Die Data Plane transportiert Aktivierungen, KV-Daten, Modellsegmente, Ergebnisse und Verifikationsinformationen.

6. Der ComputeMesh Node

Windows ist die erste Zielplattform. Der Node muss nach Installation automatisch Hardware und Netzwerkeigenschaften erkennen, benchmarken, beim Control Plane registrieren und ohne manuelle Infrastrukturkonfiguration Jobs übernehmen können.

Kategorie	Automatisch erfassen
Rechenhardware	CPU, RAM, GPU, VRAM, Compute Capability
GPU-Stack	Treiber, CUDA/ROCm/Metal, Backend-Version
Interconnect	PCIe, NVLink, Thunderbolt/RDMA, LAN
Netzwerk	Upload, Download, RTT, Jitter, Packet Loss
System	OS, Speicher, Energieoptionen
Betrieb	Temperatur, Power-Limit, Verfügbarkeit

7. Dynamisches Hardwareprofil

```
{
  "gpu": "RTX 4090",
  "vram_gb": 24,
  "llm_int4_score": 187,
  "llm_fp16_score": 91,
  "network_latency_ms": 14,
  "upload_mbps": 187,
  "availability": 0.982,
  "reliability": 0.997,
  "region": "DE",
  "power_limit_w": 260
}
```

Das Profil ist keine statische Gerätebeschreibung. Es wird kontinuierlich aus Telemetrie aktualisiert, weil Netzqualität, thermische Drosselung, Auslastung und Verfügbarkeit schwanken.

8. Benchmark-System

Ein klassischer FLOPS-Wert reicht nicht. Der Benchmark muss die tatsächlichen Engpässe moderner LLM-Inferenz messen.

Benchmark	Zweck
GEMM	Matrixmultiplikation
Attention	Attention-Kernel und Sequenzabhängigkeit

KV-Cache	Speicherzugriff und Kontextkosten
Memory Bandwidth	Gewichtstransfer
Quantized GEMM	INT4/INT8-Eignung
Prefill	Prompt-Verarbeitung
Decode	Token-Erzeugung
Embedding	Batch-Eignung
Network Transfer	Effektive Datenrate zwischen Nodes

9. Compute Points (CP)

CP ist in Version 1 ausschließlich eine technische Maßeinheit. Sie dient dazu, heterogene Hardware zu normalisieren und Scheduler-Entscheidungen sowie Preisbildung zu unterstützen. CP ist ausdrücklich kein handelbarer Token.

```

LLM Decode   82 CP
LLM Prefill  117 CP
MoE          101 CP
Embedding    149 CP
Network      91 CP
-----
Overall      106 CP

```

10. Model Registry

Jedes unterstützte Modell besitzt ein maschinenlesbares Manifest. Dieses enthält Architektur, Layer, Experten, Quantisierung, Speicherbedarf, KV-Anforderungen, Backend-Kompatibilität und erlaubte Partitionierungsstrategien.

```

MODEL: Frontier-400B
Architecture: Transformer / MoE
Layers: 128
Experts: 256
Quantization: INT4
Model Memory: 231 GB
Backends: CUDA / ROCm / Metal
Partitioning: pipeline, expert, tensor(local)

```

11. Model Sharding

Modelle werden in reproduzierbare Shards zerlegt. Shards müssen versioniert, signiert, gehasht und über einen Content-Addressed Store referenziert werden. Der Scheduler platziert zusammenhängende Blöcke bevorzugt auf Nodes mit niedriger gegenseitiger Latenz.

Embedding → Layers 0-7 → Layers 8-15 → Layers 16-23 → ... → Output

Bei MoE-Modellen werden Expertengruppen zusätzlich separat adressierbar. Das ermöglicht unabhängige Replikation und lastabhängige Platzierung.

12. Das Topologieproblem

800 GB verteiltes VRAM sind nicht dasselbe wie 800 GB lokales VRAM.

Die eigentliche technische Schwierigkeit ist Kommunikation. Für interaktive Inferenz ist die Position der Shards mindestens so wichtig wie rohe GPU-Leistung. Der Scheduler muss die gesamte Kette optimieren, nicht einzelne Nodes isoliert.

GUT: 8GB --4ms--> 8GB --7ms--> 24GB --9ms--> A100

SCHLECHT: A100(DE) --110ms--> A100(US) --180ms--> 4090(JP)

13. Scheduler

Der Scheduler ist das zentrale Optimierungssystem. Eine vereinfachte Score-Funktion kann als Ausgangspunkt dienen:

$$\text{Score} = (\text{Compute} \times \text{Reliability} \times \text{Locality} \times \text{Availability}) / (\text{Latency} \times \text{Price} \times \text{FailureRisk})$$

Später wird daraus ein Multi-Objective-Optimizer. Er minimiert Time-to-First-Token, Inter-Token-Latency, Netzwerkverkehr, Kosten und Ausfallwahrscheinlichkeit und maximiert gleichzeitig Throughput, GPU-Auslastung und Provider-Ertrag.

14. Vier Netzwerkklassen

Klasse	Verbindung	Geeignete Strategie
A - Ultra Local	PCIe, NVLink, TB/RDMA, 100/400 Gbit	Tensor Parallelism
B - Local	gleiches LAN/Rechenzentrum	Tensor + Pipeline Parallelism
C - Regional	niedrige Internetlatenz	große Layer-Blöcke / Pipeline
D - Global	hohe Latenz	Batch, Expert Parallelism, Verification

Die Klassifizierung verhindert den häufigsten Designfehler: eine Parallelisierungsstrategie zu verwenden, die nur in einem Rechenzentrum funktioniert, und sie unverändert auf das öffentliche Internet zu übertragen.

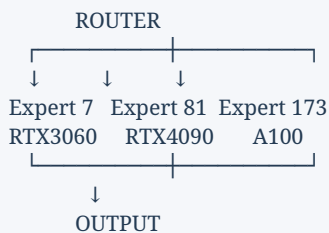
15. Dynamische Partitionierung

ComputeMesh entscheidet pro Session, ob Tensor-, Pipeline-, Expert- oder Data-Parallelism verwendet wird. Auch Mischformen sind zulässig. Die Entscheidung hängt von Modellstruktur, Node-Topologie, verfügbarer Bandbreite, Latenz und Kosten ab.

- Tensor Parallelism: nur bei sehr schneller Verbindung
- Pipeline Parallelism: für größere zusammenhängende Modellblöcke
- Expert Parallelism: besonders geeignet für MoE
- Data Parallelism: für unabhängige Requests und Batch-Workloads

16. Mixture of Experts als strategischer Schwerpunkt

MoE passt strukturell besser zu einem heterogenen Internetnetz, weil pro Token nur ein Teil der Experten aktiviert werden muss. Experten können unabhängig gehostet, repliziert und geografisch verteilt werden.



17. Langfristige Forschungsvision: Internet-Native MoE

Die langfristig stärkste Differenzierung wäre ein Modell, dessen Architektur nicht für NVLink-Cluster entworfen und anschließend ins Internet gezwungen wird, sondern von Anfang an mit Latenz, Ausfällen, heterogenen Beschleunigern und dynamischer Verfügbarkeit rechnet.

Nicht ein Rechenzentrumsmodell verteilen - ein Modell für das Internet entwerfen.

18. Eigenschaften eines Internet-Native MoE

- tausende, separat adressierbare Experten
- replizierbare Experten
- asynchrones Routing
- latenzbewusstes Expert Routing
- Fallback-Experten
- mehrere Qualitäts-/Geschwindigkeitsstufen
- Speculative Execution
- Expert Caching
- Hot-Expert-Replikation
- ressourcen- und preisbewusstes Routing

19. Hot Experts

Stark nachgefragte Experten werden automatisch repliziert. Ein Expert Placement Service beobachtet Auslastung und Request-Verteilung und erzeugt zusätzliche Kopien nahe der Nachfrage.

```
Expert 17
├─ Frankfurt
├─ Paris
├─ London
├─ New York
└─ Tokyo
```

Die Auswahl der Replik erfolgt nach Latenz, Queue-Länge, Reputation, Preis und aktueller GPU-Auslastung.

20. Neural CDN

Die passende Analogie ist ein Content Delivery Network für Modellintelligenz. Ein klassisches CDN repliziert Dateien nahe beim Nutzer. ComputeMesh repliziert Modellblöcke und Experten dort, wo sie benötigt werden.

Neural CDN = geografisch verteilte, nachfragegesteuerte Platzierung von Modellintelligenz.

21. Fehlertoleranz

Consumer-Nodes sind grundsätzlich unzuverlässiger als Datacenter-Nodes. Das System muss daher davon ausgehen, dass Nodes jederzeit verschwinden. Kritische Shards und Experten benötigen Replikate und vorbereitete Failover-Pfade.

```
Shard 27
Primary: RTX4090 Frankfurt
Replica: A100 Amsterdam
Replica: RTX3090 Munich
```

Bei einem Ausfall wird nicht das gesamte Modell neu geplant. Nur die betroffene Route bzw. Stage wird re-konfiguriert.

22. Predictive Failure und Availability Learning

ComputeMesh soll zeitliche Nutzungsmuster lernen. Ein Node, der werktags regelmäßig um 17:30 fürs Gaming gebraucht wird, darf kurz vorher keine kritischen Langzeit-Shards mehr erhalten. Diese Vorhersage reduziert Session-Abbrüche und unnötige Replikation.

23. Datenschutzklassen

Öffentliche Consumer-Nodes sind nicht automatisch für sensible Daten geeignet. Datenschutz wird deshalb als explizite Scheduling-Dimension behandelt.

Klasse	Beispiel
Public Compute	nicht sensible Workloads auf offenen Nodes

EU Verified	nur verifizierte Nodes in der EU
Datacenter Only	nur professionelle Provider
Confidential Compute	nur geeignete TEE-/Confidential-Compute-Infrastruktur

24. Sicherheit des Providers

V1 führt keinen beliebigen Kundencode auf privaten Rechnern aus.

Zugelassen sind ausschließlich signierte ComputeMesh-Worker und definierte Inference-Workloads. Keine fremde Shell, kein beliebiges Python und keine unkontrollierten Container. Erst spätere Versionen können stark isolierte allgemeine Workloads erlauben.

25. Verifikationsstufen

Level	Mechanismus
0	Benchmark + Reputation
1	Canary Jobs mit bekanntem Ergebnis
2	Random Redundancy auf zweitem Node
3	Challenge/Response und Trace-Prüfung
4	kryptographisch stärkere Proof-of-Inference-Verfahren

Das System muss zwischen Vertrauen, Kosten und Sicherheit abwägen. Eine vollständige Doppelberechnung jedes Jobs würde den ökonomischen Vorteil zerstören.

26. Adaptive Verification

Die Verifikationsrate wird risikobasiert gesetzt. Ein neuer Node wird stärker überprüft als ein langjährig zuverlässiger Node; ein hochpreisiger oder sicherheitskritischer Job stärker als ein billiger Batch-Job.

Neuer Node → 10 % Kontrolle
 10.000 korrekte Jobs → 1 % Kontrolle
 1 Mio. korrekte Jobs → 0,1 % Kontrolle
 High-value Job → bis 100 % Kontrolle

27. Reputation

Reputation ist mehrdimensional und kryptographisch an die Node-Identität gebunden. Sie soll nicht einfach gekauft werden können.

Signal	Einfluss
Uptime	Verfügbarkeit und Planbarkeit
Result correctness	Verifikations-/Canary-Ergebnisse
Performance	tatsächliche vs. versprochene Leistung
Abort rate	Abbrüche und Timeouts
Network quality	RTT, Jitter, Loss
Node age	Langzeitstabilität

```

NODE #82171
Reliability 99.997 %
Jobs      827,114
Uptime    98.4 %
Verification 99.999 %
Trust     AAA

```

28. Payment: Version 1

Version 1 verwendet Fiat-Abrechnung. Der Kunde zahlt in Euro oder einer anderen normalen Währung. ComputeMesh führt intern ein hochauflösendes Ledger; Bank-/Payment-Transaktionen erfolgen aggregiert, nicht pro Token.

```

Job #72612 +0.0027 EUR
Job #72613 +0.0041 EUR
Job #72614 +0.0019 EUR
-----
Node balance 17.28 EUR

```

29. Abrechnungslogik

Ein Request kann intern in Compute-, Netzwerk- und Verifikationskosten zerlegt werden. Die Prozentsätze sind Platzhalter und müssen durch reale Unit Economics kalibriert werden.

Komponente	Beispiel
Compute	0,00271 EUR
Network	0,00017 EUR
Verification	0,00008 EUR
Total	0,00296 EUR

Verteilung	Illustratives Modell
------------	----------------------

Provider	90 %
ComputeMesh	8 %
Reserve/Verification	2 %

30. Compute Credits

Später kann ein Provider wählen, ob er Geld oder Compute-Guthaben erhält. Dadurch entsteht ein geschlossener Nutzkreislauf: Die kleine GPU arbeitet nachts und finanziert tagsüber den Zugriff auf ein wesentlich größeres Modell.

Share small GPU → Earn compute → Run massive model

31. Kein eigener Token zu Beginn

Das Netzwerk braucht zuerst reale Nachfrage, reale Jobs und reale Abrechnung. Ein spekulativer Token würde technische Entwicklung, regulatorische Komplexität und Markenwahrnehmung unnötig belasten. Erst wenn Compute tatsächlich einen stabilen wirtschaftlichen Wert besitzt, kann über dezentrale Settlement-Mechanismen nachgedacht werden.

32. Node UI

COMPUTEMESH

Your GPU: RTX 3060 8GB

STATUS: ● Sharing

Today: €0.84

This month: €17.28

GPU: 67%

Temperature: 64°C

[STOP SHARING]

Die Oberfläche darf nicht wie Mining-Software oder ein DevOps-Tool wirken. Sie soll drei Dinge beantworten: Ist mein Node aktiv? Was verdient er? Welche großen Modelle kann ich dadurch nutzen?

33. Killer-UX: Run Massive Model

Your GPU

 8 GB

Model requires

 280 GB

Finding network compute...

8 → 16 → 40 → 72 → 152 → 287 GB

MODEL READY
[START]

Dieser Moment ist die wichtigste Consumer-Demonstration des Produkts. Der Nutzer erlebt unmittelbar, dass seine lokale Hardwaregrenze durch das Netz aufgehoben wird.

34. Entwickler-API

Neben der Desktop-App braucht ComputeMesh eine OpenAI-kompatible API, damit bestehende Anwendungen mit minimalem Integrationsaufwand wechseln können.

```
/v1/chat/completions
/v1/responses
/v1/embeddings
/v1/models
```

Später kommen ComputeMesh-spezifische Endpunkte hinzu, z. B. Privacy Tier, Deadline, Budget, Quality Target und bevorzugte Regionen.

35. Repository-Struktur

```
computemesh/
├─ README.md
├─ LICENSE
├─ SECURITY.md
├─ CONTRIBUTING.md
├─ ARCHITECTURE.md
├─ PROTOCOL.md
├─ THREAT_MODEL.md
├─ apps/
│  └─ node/
│     └─ desktop/
│        └─ dashboard/
│           └─ admin/
├─ services/
│  └─ gateway/
│     └─ scheduler/
│        └─ registry/
│           └─ billing/
│              └─ verification/
│                 └─ telemetry/
├─ runtime/
│  └─ cuda/
│     └─ llama/
│        └─ vllm/
│           └─ network/
├─ protocol/
├─ sdk/
├─ models/
├─ tests/
├─ deploy/
├─ docs/
└─ research/
```

36. Programmiersprachen und Laufzeit

Technologie	Rolle
Go	Control Plane, Scheduler, Node Daemon, Networking, Registry, Billing
C++/CUDA	performancekritische Runtime und Kernel-Integration
Python	ML-Forschung, Benchmarks, Modell-Experimente
TypeScript/React	Desktop-/Web-Oberfläche
PostgreSQL	persistente Geschäfts- und Topologiedaten

37. Datenmodell

Das Datenmodell muss von Beginn an Messbarkeit und Auditierbarkeit unterstützen. Kernobjekte:

```

users
nodes
hardware
benchmarks
models
model_shards
jobs
job_segments
payments
ledger
reputation
verification
sessions
clusters
network_metrics

```

38. Node-State-Machine

OFFLINE → CONNECTING → AUTHENTICATING → BENCHMARKING → READY

```

↓
ASSIGNED
↓
LOADING
↓
SERVING
↓
DRAINING
↓
READY

```

Errors: FAILED / QUARANTINED / BANNED

Alle Zustandsübergänge müssen idempotent, protokolliert und nach Absturz wiederherstellbar sein.

39. Job-State-Machine

CREATED → PLANNING → RESERVING → DISPATCHING → RUNNING → VERIFYING → COMPLETED → SETTLED

Recovery states: RETRY / REPLAN / FAILED / REFUNDED

Ein Job muss deterministisch nachvollziehbar sein: verwendete Modellversion, Nodes, Shards, Kosten, Verifikation und Endresultat werden eindeutig referenziert.

40. Teststrategie

- Unit Tests für Scheduler, Ledger, Manifeste und State Machines
- Integration Tests für Node <-> Scheduler <-> Runtime
- Distributed Tests mit heterogenen Geräten
- Chaos Tests mit Ausfällen und Netzproblemen
- Performance Tests mit TTFT/TPS/Traffic
- Security Tests und Supply-Chain-Prüfung
- Billing Tests mit Rundungs- und Retry-Szenarien

41. Chaos Testing

Fehler werden absichtlich erzeugt. Ziel ist nicht, dass Nodes nie ausfallen, sondern dass das System Ausfälle erwartet und korrekt behandelt.

- Node während Inferenz ausschalten
- Internetverbindung trennen
- GPU-Worker töten
- 500 ms Latenz injizieren
- Packet Loss erzeugen
- Modell-Shard beschädigen
- Scheduler neu starten
- Datenbank-Failover provozieren

Erfolgskriterium: Der Benutzer erhält möglichst trotzdem eine korrekte Antwort oder einen sauberen, abrechnungsneutralen Retry.

42. M0 - Forschung und Architektur

Zeitraumen: ca. 2 Wochen. Ziel ist kein Produkt, sondern eine belastbare technische Ausgangsbasis.

- Petals, exo und verwandte Distributed-Inference-Ansätze analysieren
- llama.cpp, vLLM, DeepSpeed, Ray und NCCL evaluieren
- gRPC/QUIC für Data Plane testen
- MoE-Routing und Modellpartitionierung vergleichen
- ARCHITECTURE.md und PROTOCOL.md v0.1 erstellen
- Messsuite für TTFT, TPS, Bandbreite und Recovery definieren

43. M1 - Zwei GPUs

Zeitraumen: Woche 3-5. Zwei Rechner führen gemeinsam ein Modell aus, das nicht vollständig auf dem kleineren Node liegt. Payment und Marketplace bleiben außen vor.

PC A: 8 GB + PC B: 24 GB → erster gemeinsamer Token

Gate 1: automatische gemeinsame Ausführung muss funktionieren.

44. M2 - Heterogener Cluster

Zeitraumen: Woche 6-8. Der Scheduler verteilt ein Modell automatisch über unterschiedliche GPU-Größen, ohne manuelles Sharding.

8 GB + 8 GB + 16 GB + 24 GB → automatic plan

Gemessen werden GPU-Idle-Zeiten, Pipeline-Bubbles, Speicherauslastung und die Sensitivität gegenüber langsamen Nodes.

45. M3 - Echtes Internet

Die Nodes stehen an unterschiedlichen physischen Orten. Jetzt entscheidet sich, welche Architekturvarianten außerhalb eines LANs tragfähig sind.

Messgröße	Bedeutung
TTFT	Startlatenz
Tokens/s	interaktive Geschwindigkeit
Traffic/token	Netzwerkökonomie
GPU utilization	Pipeline-Effizienz
Jitter sensitivity	Robustheit
Recovery time	Fehlertoleranz

46. M4 - 70B-Demonstrator

Ein Nutzer mit 8 GB startet ein 70B-Modell, das ohne Netz nicht lokal laufen könnte. ComputeMesh beschafft den fehlenden Speicher und die fehlende Rechenleistung automatisch.

Erster öffentlich verständlicher Wow-Moment: 8 GB local → 70B network model.

47. M5 - Node Failure während Chat

Während einer laufenden Session wird ein beteiligter Node absichtlich ausgeschaltet. Das System muss auf eine Replica oder einen neu geplanten Pfad wechseln. Dieser Test wird später Teil der Launch-Demonstration.

48. M6 - 100-Node Private Alpha

Ziel: mindestens 100 Nodes, mehr als 10 GPU-Typen, mehrere Regionen und kontinuierliche Telemetrie. Der Fokus liegt auf realen Verfügbarkeitsmustern und Netzwerkheterogenität.

Ziel	Richtwert
Nodes	100+
GPU-Typen	10+
Regionen	5+
Betrieb	24/7 Messung
Sessions	tausende Test-Sessions

49. M7 - Payment

Der erste reale wirtschaftliche Kreislauf wird geschlossen: Kunde A zahlt einen echten Betrag, Provider B erhält für einen korrekt abgeschlossenen und verifizierten Job eine Gutschrift.

CUSTOMER → PAYMENT → JOB → GPU → RESULT → VERIFY → PROVIDER LEDGER

50. M8 - 400B-Klasse

Ein Modell mit mehreren hundert Gigabyte Speicherbedarf läuft über ComputeMesh, während der Nutzer selbst nur eine kleine GPU besitzt. Dieses Ziel ist technisch wesentlich anspruchsvoller als der 70B-Demonstrator und sollte erst nach stabiler Topologie- und Failover-Logik verfolgt werden.

51. M9 - Public Alpha

Die Public Alpha darf nur veröffentlicht werden, wenn Installation, Benchmark, Node-Betrieb, Updatepfad und Fehlerdiagnose weitgehend automatisiert sind.

Download ComputeMesh.exe → Install → Login → Benchmark → Start

Der Nutzer darf nicht mit CUDA-Versionen, Portweiterleitungen oder manuellen Shards konfrontiert werden.

52. M10 - 10.000 GPUs

Dies ist ein Skalierungsziel, keine Prognose. Entscheidend ist nicht die reine Node-Zahl, sondern nutzbarer VRAM, regionale Dichte, Zuverlässigkeit und tatsächliche Nachfrage.

10,000 GPUs
100+ TB VRAM
20+ countries
regional density metrics
real paid workloads

53. M11 - Internet-Native MoE Research Program

Nach Validierung des Netzwerks beginnt ein eigenes Forschungsprogramm für ein Modell, das ComputeMesh-spezifische Eigenschaften nativ nutzt: Expert Placement, Ausfalltoleranz, heterogene Quantisierung und netzwerkbewusstes Routing.

54. Consumer Launch

YOU HAVE 8 GB. THE MODEL NEEDS 300 GB. RUN IT ANYWAY.

Die Kampagne zeigt nicht Architekturdiagramme, sondern eine klare Grenzüberschreitung: Der lokale Rechner ist zu klein, das Netzwerk macht das Modell dennoch verfügbar.

8 GB → 16 GB → 40 GB → 72 GB → 152 GB → 287 GB

MODEL READY

55. Provider Launch

YOUR GPU IS SLEEPING. PUT IT TO WORK.

Die Provider-Botschaft konzentriert sich auf Nutzen statt Spekulation: Compute bereitstellen, Compute-Guthaben oder Geld verdienen und im Gegenzug größere Modelle nutzen.

56. Der große Launch-Satz

THE INTERNET IS YOUR GPU

Unterzeile: Run AI models bigger than your computer by connecting to GPUs around the world.

Der Satz eignet sich als Leitidee, muss vor Marken- und Domainentscheidung allerdings juristisch und markenrechtlich geprüft werden.

57. Open-Source-Strategie

Offenheit schafft Vertrauen und erleichtert Adoption. Gleichzeitig muss ein wirtschaftlicher Burggraben entstehen.

Offen

Kommerzieller Burggraben

Protocol	Netzwerkliquidität
Node	Scheduler-Qualität
SDK	Marketplace
Model Manifests	Enterprise/SLA
Benchmark Spec	Verification-Infrastruktur
Referenzimplementierungen	Brand, Support, Routing-Daten

58. IP- und Patentstrategie

Vor Veröffentlichung technisch neuer Mechanismen ist eine professionelle Prior-Art-Recherche sinnvoll. Nicht die breite Idee „dezentrale KI“ ist schutzwürdig, sondern konkrete technische Verfahren mit nachweisbarer Neuheit und erfinderischem Schritt.

- Adaptive topology construction
- Heterogeneous AI routing
- Economic-aware partitioning
- Adaptive verification
- Predictive shard replication
- Internet-aware expert routing
- Compute-market-aware inference
- Dynamic expert placement

59. Team

Rolle	Verantwortung
Founder / Product	Vision, Prioritäten, Finanzierung, Partnerschaften
Distributed Systems Engineer	Scheduler, Control/Data Plane, Failover
ML Systems / CUDA	Runtime, Sharding, Kernel, Benchmarks
Windows / Go Engineer	Consumer Node, Installer, Update, Hardware-Erkennung
Security Engineer	Sandbox, Identität, Verification, Supply Chain
Frontend/Product	Node UI, Dashboard, API Console
SRE/Infra	Betrieb und Observability
Research	Internet-Native MoE

Für M0-M3 ist ein kleines Kernteam aus 2-4 sehr starken Entwicklern realistischer und effizienter als ein großes Team.

60. Aufgaben des Founders

1. Produktthese schützen: „One click → massive model → network compute“.
2. Technische Ziele messbar machen und Scope begrenzen.
3. Starkes Distributed-Systems-/ML-Systems-Kernteam gewinnen.
4. IP, Entwicklerverträge, Marke und Repository-Rechte sauber organisieren.
5. Frühe Provider für heterogene Hardware gewinnen.
6. Parallel erste Nutzer und AI-Workloads identifizieren.
7. Nicht vor dem technischen Beweis zu früh auf Token, große Finanzierung oder Marketing fokussieren.
8. Nach M3/M4 mit Demo und Messwerten gezielt Finanzierung/Partnerschaften ansprechen.

61. Wichtigste KPIs

KPI	Warum entscheidend
TTFT	Wie schnell startet die Antwort?
Tokens/s	Ist die Nutzung interaktiv brauchbar?
Effective VRAM	Wie viel verteilter Speicher ist real nutzbar?
Traffic/token	Wie teuer ist das Internet-Sharding?
GPU utilization	Wie groß sind Pipeline-Bubbles?
Failure recovery	Wie schnell überlebt die Session Node-Ausfälle?
Cost/token	Ist das Modell wirtschaftlich konkurrenzfähig?
Availability	Wie stabil ist das virtuelle System?

62. Das wichtigste technische Ziel

8-GB-PC betreibt über ComputeMesh ein Modell mit >200 GB Speicherbedarf bei interaktiv brauchbarer Geschwindigkeit.

Illustrative Performance-Stufen: 0,5 Token/s ist Forschungsdemonstrator; etwa 5 Token/s wird praktisch interessant; 20+ Token/s wäre für viele interaktive Szenarien sehr stark. Diese Werte sind keine Garantie und hängen von Modell, Kontext, Quantisierung und Netzwerktopologie ab.

63. Go/No-Go-Entscheidungen

Gate	Messbarer Nachweis	Entscheidung
------	--------------------	--------------

Gate 1	Heterogene GPUs führen Modell automatisch gemeinsam aus	weiter / Architekturwechsel
Gate 2	Echte Internetverbindung erreicht brauchbare TTFT/TPS	interaktiv / regional / Batch-MoE
Gate 3	Cost/Token inkl. Verification und Payment konkurrenzfähig	Marketplace skalieren / Positionierung ändern

64. Was als Erstes gebaut wird

Nicht Firma, Coin oder große Marketingseite. Zuerst vier kontrollierte technische Experimente:

Experiment	Aufbau	Messung
A	2 GPUs über LAN	Baseline für Sharding
B	2 GPUs über Internet	Latenz und Traffic
C	4 heterogene GPUs	automatische Partitionierung
D	Node während Inferenz entfernen	Recovery/Fault Tolerance

Für alle Experimente werden TTFT, Tokens/s, Bandbreite, VRAM, GPU-Auslastung, Jitter und Recovery Time automatisiert protokolliert.

65. Endbild

Ein Schüler stellt eine 8-GB-GPU bereit. Eine Universität hat nachts A100-Kapazität frei. Ein Gamer besitzt eine High-End-GPU. Eine Mining-Farm betreibt hunderte ältere RTX-Karten. Ein Rechenzentrum bietet H100-Restkapazität. Alle sprechen dasselbe Protokoll.

```

CONSUMER GPUs + WORKSTATIONS + MINING RIGS + A100/H100 CLUSTERS
    ↓
MEMORY + COMPUTE + ROUTING + REPLICATION
    ↓
VERIFICATION + PAYMENT
    ↓
[ RUN FRONTIER MODEL ]

```

Die eigentliche Erfindung wäre eine verteilte Hardware- und Ausführungsschicht des Internets für künstliche Intelligenz.

B. Konkreter 90-Tage-Execution-Plan

Zeitraum	Primärziel	Deliverables
----------	------------	--------------

Tag 1-14	Architektur & Benchmark	Repo, ADRs, Benchmark Harness, 2-Node Lab
Tag 15-35	M1 Zwei GPUs	Remote Shard Load, Token Pipeline, Messwerte
Tag 36-56	M2 Heterogen	Planner, Hardware Profiles, Auto Partition
Tag 57-70	M3 Internet	WAN Tests, QUIC/gRPC Vergleich, Latenzmodell
Tag 71-80	M4 70B Demo	8-GB-Client + Network Model
Tag 81-90	M5 Failover	Replica, Replan, Demo Recording

Definition of Done nach 90 Tagen

- Eine reproduzierbare Demo läuft auf mindestens drei unterschiedlichen GPU-Klassen.
- Ein Modell größer als die lokale GPU-Grenze wird automatisch über das Netzwerk ausgeführt.
- Alle wesentlichen Performance-KPIs werden automatisch erfasst.
- Node-Ausfall wird entweder transparent abgefangen oder führt zu einem sauberen Retry ohne falsche Abrechnung.
- ARCHITECTURE.md, PROTOCOL.md und THREAT_MODEL.md sind aktuell.
- Eine belastbare Entscheidung liegt vor, ob der Fokus interaktive Pipeline-Inferenz, MoE/Expert Routing oder Batch-Compute sein soll.

C. Launch-Readiness-Checkliste

- Installer signiert und reproduzierbar
- Auto-Update mit Rollback
- GPU-/Treiberprüfung robust
- Node darf Hostsystem nicht destabilisieren
- Power-/Temperaturlimits funktionieren
- Privacy Tier wird technisch erzwungen
- Payment-Testfälle vollständig
- Ledger ist auditierbar
- Canary-/Verification-System aktiv
- Crash Reporting datensparsam
- Öffentliche Statusseite
- Dokumentierte Mindesthardware
- Klare Einnahmen- und Kostenanzeige
- Keine Renditeversprechen
- Support- und Incident-Prozess
- Security Disclosure / SECURITY.md
- Marke/Domain geprüft
- Datenschutz-/AGB-/Zahlungsmodell rechtlich geprüft

D. Entscheidungsprinzipien

1. Performance vor Dezentralisierungsromantik: Jede verteilte Komponente muss messbar Nutzen bringen.
2. Kommunikation ist ein First-Class-Resource: Bandbreite, RTT und Jitter werden wie VRAM und FLOPS geplant.
3. Model-aware Scheduling statt generischer VM-Marktplatz.
4. MoE und Expert Placement früh untersuchen, wenn dichte Modelle über WAN nicht ausreichend performant sind.
5. Consumer-Sicherheit vor Workload-Flexibilität: V1 nur kontrollierte Inferenz.
6. Reale Nachfrage vor Tokenisierung.
7. Messwerte vor Marketingbehauptungen.
8. Offenes Protokoll, aber gezielt geschütztes technisches IP dort, wo echte Neuheit besteht.

E. Projekt-Nordstern

THE INTERNET IS YOUR GPU

Der erste Beweis ist nicht „10.000 Nodes“. Der erste Beweis ist: Ein kleiner Rechner startet zuverlässig ein großes Modell über fremde, heterogene GPUs - und die Nutzererfahrung fühlt sich wie ein einziger Computer an.